APUNTES GIT CARLES

GITHUB BASICO

```
git config --global user.name "Nom Cognom"  git
config --global user.email "correu@exemple.com"


gh auth login --with-token < auth.txt
gh auth setup-git

mkdir -p ~/examen
cd ~/examen
git init
 git remote add origin   https://github.com/TU_USUARIO/exercici-
1.git
git remote -v


echo "# Respostes — Tu Nombre Apellido" >
respostes.md git add respostes.md git commit -m "Primer
commit"
/
touch respostes.md
nano resposted.md : # Respostes — Tu Nombre Apellido"
 git add respostes.md
git commit -m "Primer commit"
 git push -u origin main

echo "Tu Nombre Apellido" > Readme.md

git add Readme.md
 git commit -m "Segon
commit"
git push

git log --oneline > log.txt

git add log.txt
git commit -m "Tercer
commit"
 git push


git log --oneline
```

Mas Comandos

```
cd ~/examen
mkdir exercici-2
cd exercici-2
git init

git remote add origin https://github.com/TU_USUARIO/exercici-2.git

cat > index.html <<'EOF'
<!DOCTYPE html>
<html lang="ca">
<head>
    <meta charset="utf-8">
    <title>Portfoli de Tu Nombre Apellido</title>
    <link rel="stylesheet" href="estil.css">
</head>
<body>
    <h1>Portfoli de Tu Nombre Apellido</h1>
    <p>Pàgina publicada amb GitHub Pages.</p>
</body>
</html>
EOF


git add index.html
git commit -m "Afegeix pàgina
principal"

echo "# Capçalera" > Readme.md  / nano Readme.md

git add Readme.md
 git commit -m "Afegeix
readme"
git push -u origin main
```

GITIGNORE

```
mkdir exercici
touch .gitignore
 nano .gitignore
          logs/
git init
 git remote add origin https://github.com/TU_USUARIO/exercici-
3.git
git add .gitignore
git commit -m "Afegeix .gitignore"
git push -u origin main
git add script.py
git commit -m "Afegeix script"
git push
```

GITHUB PAGES

En GitHub crea un repositorio público llamado:

TU_USUARIO.github.io
tener un html

git init -b main git remote add origin
https://github.com/TU_USUARIO/TU_USUARIO.github.io.git git add .
git commit -m "Afegeix pàgina web"
git push -u origin main

Activar GitHub Pages

Entra en tu repositorio y sigue esta ruta:

1. Settings.
2. Pages.
3. En Build and deployment, selecciona Deploy from a branch.
4. En Branch, selecciona main.
5. En la carpeta, selecciona / (root).
6. Pulsa Save.

Espera a que se publique la web. La dirección será:
https://TU_USUARIO.github.io/

nano Readme.md
git add README.md
git commit -m "Afegeix el meu
nom"
git push

GITHUB ACTIONS

Crea en GitHub un repositorio público llamado x

Entra en la carpeta del proyecto y configura Git:

cd ruta/a/exercici-5

git init -b main
git remote add origin https://github.com/TU_USUARIO/exercici-5.git
mkdir -p .github/workflows
Crea el archivo .github/workflows/compila.yml con este contenido:
nano .github/workflows/compila.yml

```
name: Tests unitaris

on:   push:
branches:
- main

jobs:   test-
win:
    runs-on: windows-latest
steps:
- name: Checkout del repositori
  uses: actions/checkout@v6

- name: Configurar Python
  uses: actions/setup-python@v6
  with:
        python-version: '3.12'

- name: Executar tests         run:
  python -m unittest -v

  test-linux:
    runs-on: ubuntu-latest
steps:
- name: Checkout del repositori
  uses: actions/checkout@v6
```

```
- name: Configurar Python
  uses: actions/setup-python@v6
  with:
        python-version: '3.13'

- name: Executar tests         run:
  python -m unittest -v

  test-mac:
    runs-on: macos-latest
steps:
- name: Checkout del repositori
       uses: actions/checkout@v6

- name: Configurar Python
  uses: actions/setup-python@v6
  with:
        python-version: '3.13'

- name: Executar tests         run:
  python -m unittest -v
```

```
git add .
git commit -m "Afegeix tests amb GitHub Actions"
git push -u origin main
```

COMANDOS UTILES

1. Listar y Crear Ramas

- `git branch`
  - *Qué hace:* Lista todas las ramas locales de tu repositorio. La rama actual aparecerá marcada con un asterisco (*).
- `git branch -a`
  - *Qué hace:* Muestra todas las ramas, tanto las locales como las remotas (del servidor).
- `git branch <nombre-de-rama>`

- *Qué hace:* Crea una nueva rama local basada en el estado actual de tu rama activa, pero sin cambiar a ella automáticamente.

2. Cambiar de Rama

- `git checkout <nombre-de-rama>`
  - *Qué hace:* Cambia tu directorio de trabajo a la rama especificada (método clásico).
- `git switch <nombre-de-rama>`
  - *Qué hace:* Es el comando moderno recomendado para cambiar de rama de forma más intuitiva.
- `git checkout -`
  - *Qué hace:* Te regresa instantáneamente a la última rama en la que estabas trabajando (muy útil para alternar entre dos ramas).

3. Crear y Cambiar al Mismo Tiempo

- `git checkout -b <nombre-de-rama>`
  - *Qué hace:* Crea una nueva rama y se cambia a ella de inmediato (método clásico).
- `git switch -c <nombre-de-rama>`
  - *Qué hace:* La alternativa moderna para crear una rama y saltar a ella en un solo paso.

4. Renombrar y Eliminar Ramas

- `git branch -m <nuevo-nombre>`
  - *Qué hace:* Renombra la rama local en la que te encuentras actualmente.
- `git branch -d <nombre-de-rama>`
  - *Qué hace:* Elimina una rama local de forma segura (Git te advertirá si la rama tiene cambios pendientes de fusión).
- `git branch -D <nombre-de-rama>`
  - *Qué hace:* Forzar la eliminación de una rama local, incluso si contiene cambios no fusionados (¡cuidado con perder código!).

5. Fusionar y Rebasar Ramas

- `git merge <nombre-de-rama>`
  - *Qué hace:* Fusiona los cambios de la rama especificada dentro de la rama en la que te encuentras actualmente.
- `git rebase <nombre-de-rama>`
  - *Qué hace:* Reubica los commits de tu rama actual sobre la base de otra rama, manteniendo un historial lineal más limpio.

6. Gestión de Ramas Remotas

- `git push origin <nombre-de-rama>`
  - *Qué hace:* Sube tu rama local al repositorio remoto (`origin`) para compartirla con tu equipo.
- `git push -u origin <nombre-de-rama>`
  - *Qué hace:* Sube la rama y además establece el vínculo (`upstream`), permitiendo que en el futuro solo uses `git push` o `git pull` sin especificar el nombre.

- ❿ `git push origin --delete <nombre-de-rama>`
  - ❿ *Qué hace:* Elimina una rama directamente del repositorio remoto.
- ❿ `git fetch --prune`

git reset
Quita cambios del área de preparación o mueve la referencia de la rama, según las opciones utilizadas. git revert COMMIT
Crea un nuevo commit que deshace los cambios de otro commit.
git
init
Crea un repositorio Git en la carpeta actual.
git clone URL
Descarga una copia de un repositorio remoto.
git status
Muestra el estado de los archivos y los cambios pendientes.
git add archivo
Añade un archivo al área de preparación.
git add .
Añade los cambios de la carpeta actual.
git commit -m "mensaje"
Guarda los cambios preparados en el historial con un mensaje.
git diff
Muestra las diferencias entre versiones o cambios.
git log
Muestra el historial de commits.
git log -n
Muestra los últimos commits; n es el número que quieres ver.
git rm archivo
Elimina un archivo del proyecto y prepara su eliminación para el commit.
.gitignore
Archivo donde indicas qué archivos o carpetas Git debe ignorar.
Git merge : fusionar ramas

estructura ::

```
name: Tests unitaris

on:   push:
branches:
- main

jobs:
  test-win:
    runs-on: windows-latest
steps:
```

```
- uses: actions/checkout@v6       -
  uses: actions/setup-python@v6
  with:
        python-version: '3.12'
- run: python -m unittest -v

  test-linux:
    runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v6       -
  uses: actions/setup-python@v6
  with:
        python-version: '3.13'
- run: python -m unittest -v

  test-mac:
    runs-on: macos-latest
    steps:
- uses: actions/checkout@v6       -
  uses: actions/setup-python@v6
  with:
        python-version: '3.13'
- run: python -m unittest -v




name: Monedes

on:
push:
    branches: [ main ]

jobs:
  create-list:
    runs-on: ubuntu-latest

    steps:
- name: Checkout del repo
  uses: actions/checkout@v4

- name: Configurem python
  uses: actions/setup-python@v7
  with:
        python-version: '3.12'
```

```
- name: Preparem l'entorn amb els
  paquets          run: pip install -r
  requirements.txt

- name: Correm l'script          run:
  python ./euro.py
```